

Arquitetura de Computadores

Tópico 0 – Computação: Abstração, Tecnologia,
Performance e Power

Ideias que Devemos Seguir

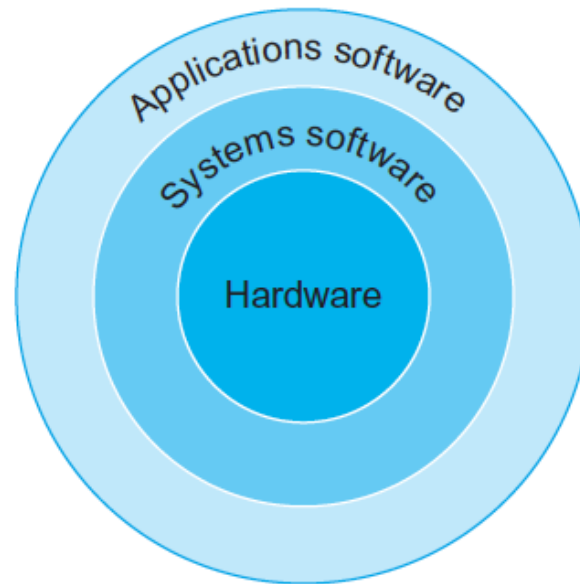
- Projetar pensando na Moore`s Law
 - Gordon Moore, em 1965, previu que os recursos (transistores) em circuitos integrados iria dobrar a cada 18-24 meses
 - Em meados de 2010 esta previsão deixou de ser verdade.
- Usar a abstração para simplificar o projeto
 - Desenvolver projeto em alto nível de abstração ganha-se em produtividade mas perde-se em performance
- Performance via Paralelismo
 - Programar pensando quais operações podem ser realizadas em paralelo, em qualquer nível de operação
- Performance via Pipelining
 - Separar o problema em partes e dividi-lo em várias pessoas

Ideias que Devemos Seguir

- Performance via Predição
 - Executar operações antes de saber se realmente serão necessárias de serem executadas
- Hierarquia de Memória
 - Utilizar a hierarquia de memória ao seu favor para ganhar performance

Abstracção: Abaixo do Programa

- Computação é composta de diversas camadas
- Abstracção traz produtividade para desenvolvedores de software



Abstracção: Fluxo de Geração

- Fluxo de geração de um executável

High-level
language
program
(in C)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
    multi $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

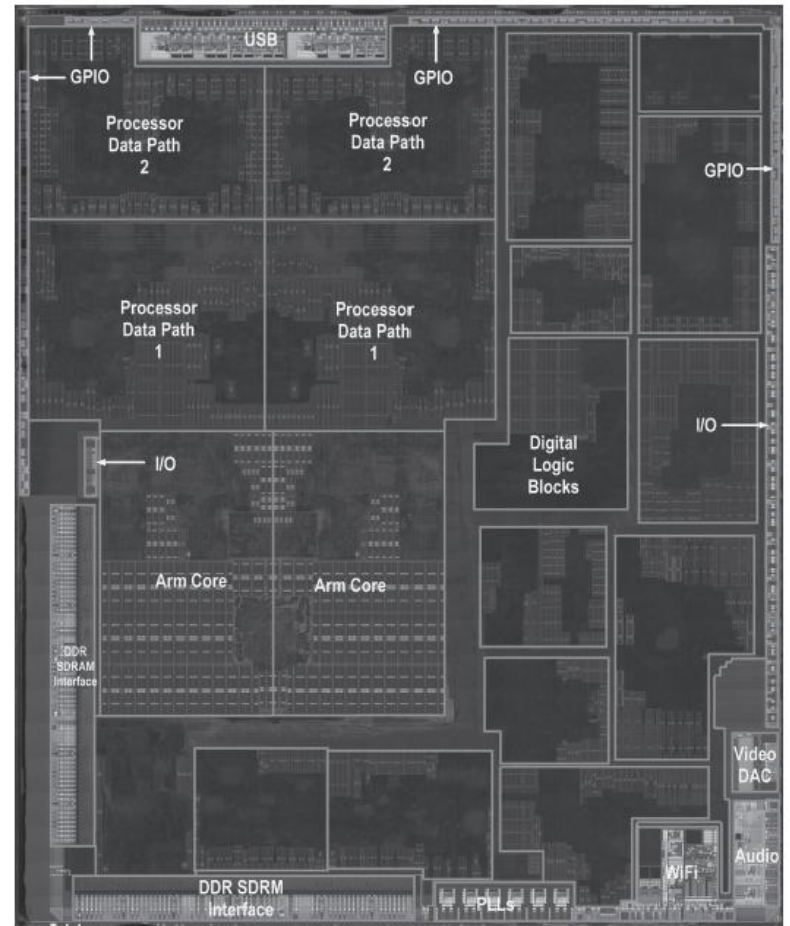
Assembler

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
101011100001001000000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```

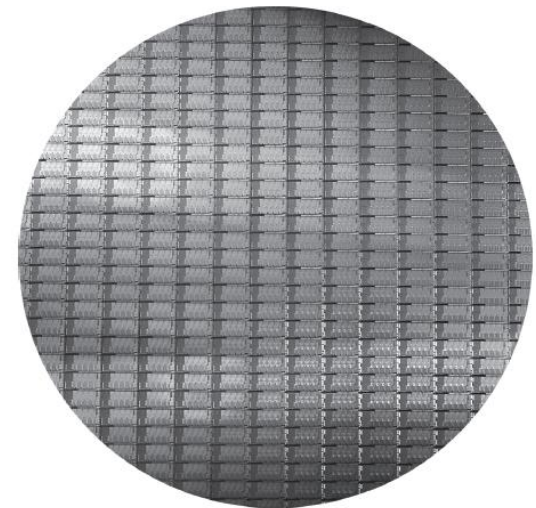
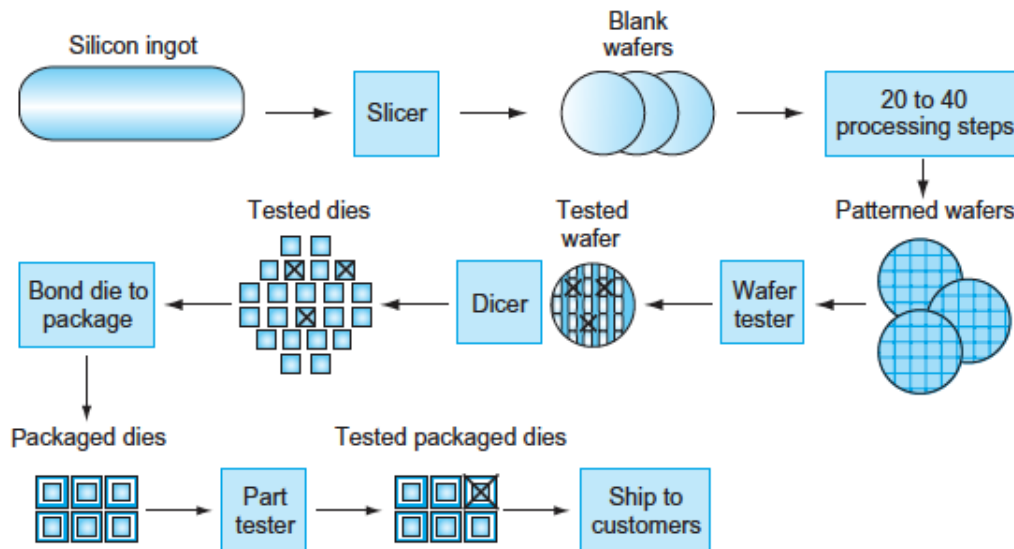
Unboxing

- Ipad



Manufatura de um Circuito Integrado

- Baseado em Silício, Semicondutor encontrado em areia
- A partir de um processo químico, pequenas áreas podem se tornar:
 - isolantes,
 - condutoras ou
 - isolantes/condutoras dependendo das condições elétricas



Definindo Performance

- Quando você afirma que um computador tem melhor performance que outro, o que você quer dizer?
- Vamos fazer uma analogia:

Airplane	Passenger capacity	Cruising range (miles)	Cruising speed (m.p.h.)	Passenger throughput (passengers × m.p.h.)
Boeing 777	375	4630	610	228,750
Boeing 747	470	4150	610	286,700
BAC/Sud Concorde	132	4000	1350	178,200
Douglas DC-8-50	146	8720	544	79,424

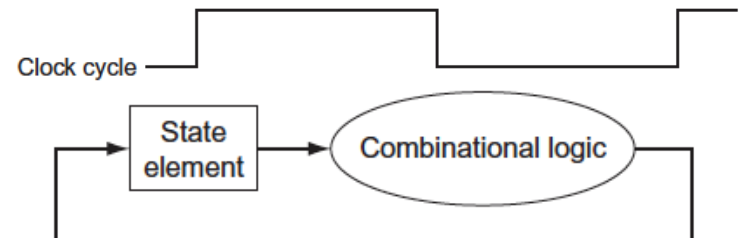
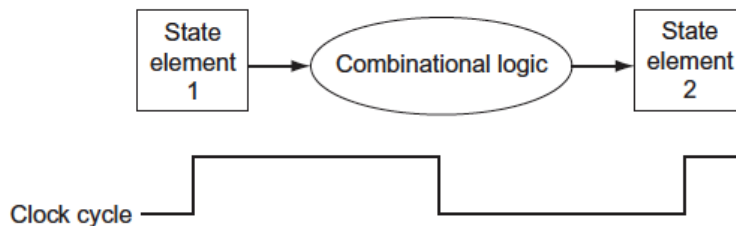
- Você está interessado em:
 - execution time- tempo para um programa executar
 - throughput- tempo para diversos programas executarem

Definindo Performance

- Usuário está interessado em ter a resposta mais rápida possível do seu programa (execution time)
- Sistema está interessado em otimizar a execução de vários programas (throughput)
- $\text{CPU time} = \text{user CPU time} + \text{system CPU time}$
- Temos de considerar tempo para realizar tarefas de I/O

Definindo Performance

- Métricas para medir performance
 - *Clock period (clock cycle) – T_c* – tempo para completar uma onda completa
 - Também é definido como o inverso da *Frequência (F)*
 - *A função do clock é controlar e evitar a imprevisibilidade do circuito pelo controle do momento onde os valores devem ser lidos e escritos (p.e. edge triggered methodology – rising edge)*

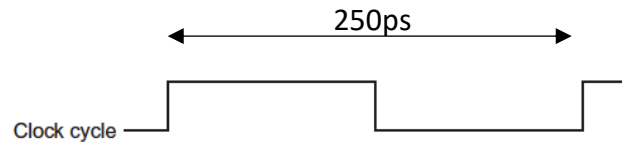


Definindo Performance

- Métricas para medir performance
 - *Clock period (clock cycle) - T_c* – tempo para completar uma onda completa
 - Também é definido como o inverso da *Frequência (F)*

Por exemplo:

$$T_c = 250 \text{ ps}$$



$$F = \frac{1}{T_c} = \frac{1}{250 \text{ ps}} = \frac{1}{250 * 10^{-12} \text{ s}}$$

$$F = 0,004 * 10^{12} \text{ Hz}$$

$$F = 4 * 10^9 \text{ Hz} = 4 \text{ GHz}$$

Metric System Prefixes			
Prefix	Symbol	Multiplier (Scientific Notation)	Multiplier
Exa	E	10^{18}	1,000,000,000,000,000,000
Peta	P	10^{15}	1,000,000,000,000,000
Tera	T	10^{12}	1,000,000,000,000
Giga	G	10^9	1,000,000,000
Mega	M	10^6	1,000,000
Kilo	k	10^3	1,000
Hecto	h	10^2	100
Deka	da	10^1	10
Deci	d	10^{-1}	0.1
Centi	c	10^{-2}	0.01
Milli	m	10^{-3}	0.001
Micro	μ	10^{-6}	0.000,001
Nano	n	10^{-9}	0.000,000,001
Pico	p	10^{-12}	0.000,000,000,001
Femto	f	10^{-15}	0.000,000,000,000,001
Atto	A	10^{-18}	0.000,000,000,000,000,001

Definindo Performance

- Métricas para medir performance
 - *Execution time of a program –ET* – tempo para completar a execução de todas as instruções do programa

$$ET = T_c * \text{Número total de ciclos (NTC)}$$

$$NTC = \text{Número total de instruções executadas (NTIE)} * \text{Número médio de ciclos por instrução (CPI)}$$

$$ET = T_c * NTIE * CPI$$

Definindo Performance

- Métricas para medir performance
 - *Execution time of a program –ET* – tempo para completar a execução de todas as instruções do programa

Por exemplo:

$$T_c = 250 \text{ ps} = 250 * 10^{-12} \text{ segundos}$$

$$NTIE = 1 \text{ milhão } (1 * 10^6 \text{ instruções})$$

$$CPI = 1$$

$$ET = T_c * NTIE * CPI$$

$$ET = 250 * 10^{-12} \text{ segundos} * 1 * 10^6 \text{ instruções} * 1$$

$$ET = 250 * 10^{-6} \text{ segundos}$$

$$ET = 0,00025 \text{ segundos}$$

Definindo Performance

- Cenário

- Compilador necessita gerar instruções para um pedaço de um código em C
 - Existem 2 sequências de instruções possíveis que refletem a lógica do código em C

Code sequence	Instruction counts for each instruction class		
	A	B	C
1	2	1	2
2	4	1	1

- Entretanto, cada instrução tem um número de ciclos para serem executadas

	CPI for each instruction class		
	A	B	C
CPI	1	2	3

- Qual a sequência que fornece maior performance para executar esta parte do código?

Definindo Performance

- Cenário

- Sequência 1

- 2 instruções A (CPI= 1)
 - 1 instrução B (CPI= 2)
 - 2 instruções C (CPI=3)

$$\text{Numero total de ciclos}(NTC) = \sum(CPI * numInstr)$$

$$NTC = 2 \times 1 + 1 \times 2 + 2 \times 3 = 10 \text{ ciclos}$$

$$CPI = NTC / numTotalInstr$$

$$CPI = 10 / 5 = 2$$

Supondo que o processador tem uma frequência de 4GHz. Qual tempo de execução de cada sequência de instrução?

$$ET = TC * NTIE * CPI$$

$$ET = 250ps * 5instr * 2 = 2500ps$$

- Sequência 2

- 4 instruções A (CPI= 1)
 - 1 instrução B (CPI= 2)
 - 1 instrução C (CPI=3)

$$NTC = 4 \times 1 + 1 \times 2 + 1 \times 3 = 9 \text{ ciclos}$$

$$CPI = NTC / numTotalInstr$$

$$CPI = 9 / 6 = 1.5$$

$$ET = 250ps * 6instr * 1.5 = 2250ps$$

$$Speedup = \frac{2500}{2250} = 1.10 = 10\%$$

The Power Wall

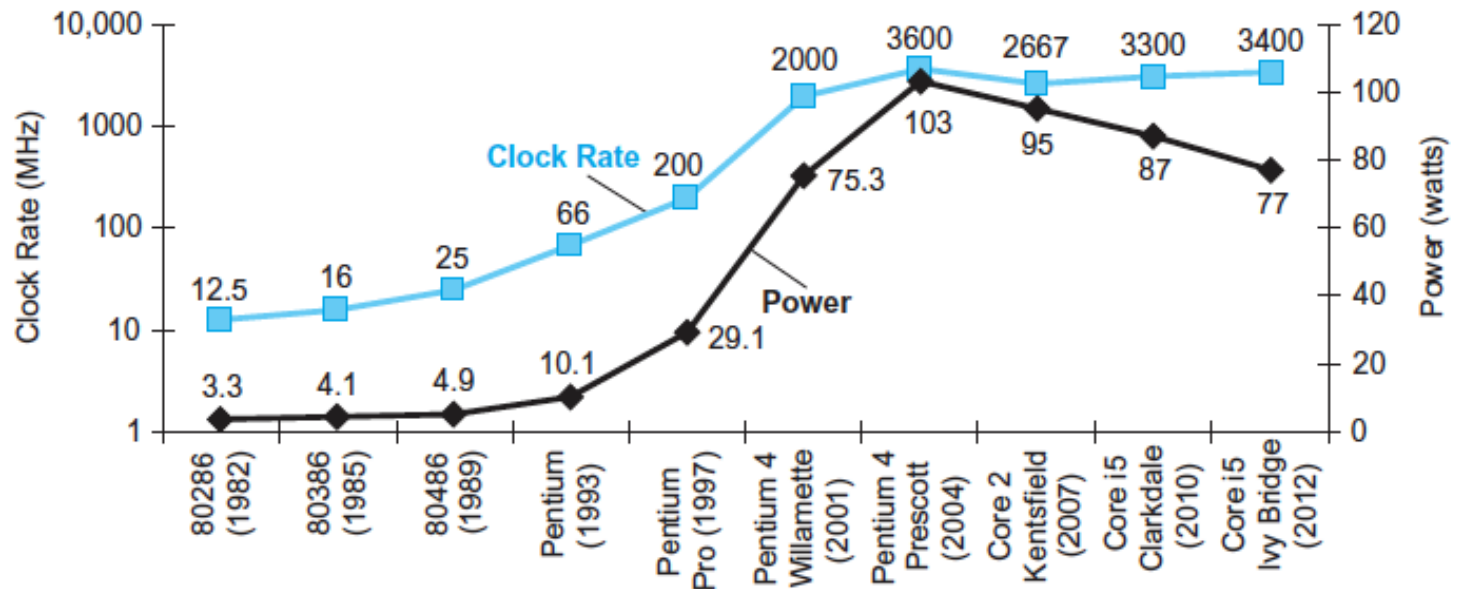
- Potência na Tecnologia CMOS é dividida em:
 - Estática (leakage) – Potência consumida quando o circuito não está chaveando
 - Dinâmica – Potência consumida no chaveamento dos transistores
- Potência dinâmica na Tecnologia CMOS é dada por:

$P = \frac{1}{2} \text{CapacitiveLoad} * \text{Voltage}^2 * \text{Frequency}$, onde:

- CapacitiveLoad - depende do número de transistores conectados a uma saída (fanout) e da tecnologia, o que determina a capacitância dos fios e transistores
- Voltage – depende da tecnologia
- Frequency – depende do circuito

The Power Wall

- Crescimento da Frequência vs Power
 - Se $P = \frac{1}{2} \text{CapacitiveLoad} * \text{Voltage}^2 * \text{Frequency}$
 - Como é explicado que a Freq do Core i5 é 272 vezes maior que a do 80286 mas a potência é somente 23 vezes maior?



Exercícios

1.5 [4] <§1.6> Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

- Which processor has the highest performance expressed in instructions per second?
- If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
- We are trying to reduce the execution time by 30% but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

1.6 [20] <§1.6> Consider two different implementations of the same instruction set architecture. The instructions can be divided into four classes according to their CPI (class A, B, C, and D). P1 with a clock rate of 2.5 GHz and CPIs of 1, 2, 3, and 3, and P2 with a clock rate of 3 GHz and CPIs of 2, 2, 2, and 2.

Given a program with a dynamic instruction count of $1.0E6$ instructions divided into classes as follows: 10% class A, 20% class B, 50% class C, and 20% class D, which implementation is faster?

- What is the global CPI for each implementation?
- Find the clock cycles required in both cases.

Exercícios

1.7 [15] <§1.6> Compilers can have a profound impact on the performance of an application. Assume that for a program, compiler A results in a dynamic instruction count of $1.0E9$ and has an execution time of 1.1 s, while compiler B results in a dynamic instruction count of $1.2E9$ and an execution time of 1.5 s.

- a. Find the average CPI for each program given that the processor has a clock cycle time of 1 ns.
- b. Assume the compiled programs run on two different processors. If the execution times on the two processors are the same, how much faster is the clock of the processor running compiler A's code versus the clock of the processor running compiler B's code?
- c. A new compiler is developed that uses only $6.0E8$ instructions and has an average CPI of 1.1. What is the speedup of using this new compiler versus using compiler A or B on the original processor?

Exercícios

1.8 The Pentium 4 Prescott processor, released in 2004, had a clock rate of 3.6 GHz and voltage of 1.25 V. Assume that, on average, it consumed 10 W of static power and 90 W of dynamic power.

The Core i5 Ivy Bridge, released in 2012, had a clock rate of 3.4 GHz and voltage of 0.9 V. Assume that, on average, it consumed 30 W of static power and 40 W of dynamic power.

1.8.1 [5] <§1.7> For each processor find the average capacitive loads.

1.8.2 [5] <§1.7> Find the percentage of the total dissipated power comprised by static power and the ratio of static power to dynamic power for each technology.

1.8.3 [15] <§1.7> If the total dissipated power is to be reduced by 10%, how much should the voltage be reduced to maintain the same leakage current? Note: power is defined as the product of voltage and current.

Exercícios

1.9 Assume for arithmetic, load/store, and branch instructions, a processor has CPIs of 1, 12, and 5, respectively. Also assume that on a single processor a program requires the execution of $2.56E9$ arithmetic instructions, $1.28E9$ load/store instructions, and 256 million branch instructions. Assume that each processor has a 2 GHz clock frequency.

Assume that, as the program is parallelized to run over multiple cores, the number of arithmetic and load/store instructions per processor is divided by $0.7 \times p$ (where p is the number of processors) but the number of branch instructions per processor remains the same.

1.9.1 [5] <§1.7> Find the total execution time for this program on 1, 2, 4, and 8 processors, and show the relative speedup of the 2, 4, and 8 processor result relative to the single processor result.

1.9.2 [10] <§§1.6, 1.8> If the CPI of the arithmetic instructions was doubled, what would the impact be on the execution time of the program on 1, 2, 4, or 8 processors?

1.9.3 [10] <§§1.6, 1.8> To what should the CPI of load/store instructions be reduced in order for a single processor to match the performance of four processors using the original CPI values?

1.12 Section 1.10 cites as a pitfall the utilization of a subset of the performance equation as a performance metric. To illustrate this, consider the following two processors. P1 has a clock rate of 4 GHz, average CPI of 0.9, and requires the execution of $5.0\text{E}9$ instructions. P2 has a clock rate of 3 GHz, an average CPI of 0.75, and requires the execution of $1.0\text{E}9$ instructions.

1.12.1 [5] <§§1.6, 1.10> One usual fallacy is to consider the computer with the largest clock rate as having the largest performance. Check if this is true for P1 and P2.

1.12.2 [10] <§§1.6, 1.10> Another fallacy is to consider that the processor executing the largest number of instructions will need a larger CPU time. Considering that processor P1 is executing a sequence of $1.0\text{E}9$ instructions and that the CPI of processors P1 and P2 do not change, determine the number of instructions that P2 can execute in the same time that P1 needs to execute $1.0\text{E}9$ instructions.

1.12.3 [10] <§§1.6, 1.10> A common fallacy is to use MIPS (millions of instructions per second) to compare the performance of two different processors, and consider that the processor with the largest MIPS has the largest performance. Check if this is true for P1 and P2.